

# Ontology Based Test Case Generation for Black Box Testing

Sami Ul Haq<sup>1</sup>, Usman Qamar<sup>2</sup>

<sup>1,2</sup> National University of Sciences and Technology, Islamabad

[sami.ulhaq@ceme.nust.edu.pk](mailto:sami.ulhaq@ceme.nust.edu.pk)<sup>1</sup>, [usmanq@ceme.nust.edu.pk](mailto:usmanq@ceme.nust.edu.pk)<sup>2</sup>

## ABSTRACT

Software systems are not considered complete unless properly tested and verified. In existing literature, a growing interest on establishment of automated testing techniques has been observed. However, tedious manual process of test case generation largely depends upon domain knowledge and formalized representation of user requirements. The advent of semantic web engineering has led the foundation for developing ontologies as a mean to express information and knowledge semantics regarding particular domain efficiently. In software testing, ontologies can be significantly helpful to automate testing phase as they encode domain knowledge in machine interpretable format. We have proposed automatic test case generation framework that involves ontology-based requirement specification and learning based methods for conducting black box testing. Our approach integrates knowledge-based system (ontology) with learning-based testing algorithm to automate: generation of test cases, test execution and test verdict construction. Proposed framework includes, requirement ontology to formalize requirement specification, Dialogue Manager that enables selection of available test cases and Learning Based Testing to generate counter examples of test cases through system learning. The contribution of this paper is to enable 1) requirement elicitation and specification using ontologies 2) test data selection from existing ontologies and 3) automatic test case generation from existing test cases. To represent the applicability of this research, ontology for requirement elicitation and specification is developed. Framework proposed in this research paper is an effort to provide software testing tools to save time, cost and efforts during test design phase.

## CCS Concepts

• Information systems~Ontologies • Software and its engineering~Specification languages • Software and its engineering~Software development methods

## Keywords

Ontologies; Black box testing; Requirement Engineering; Test case generation; Learning based testing.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Request permissions from [Permissions@acm.org](mailto:Permissions@acm.org).

ICEIT 2019, March 2–4, 2019, Cambridge, United Kingdom

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6267-2/19/03...\$15.00

<http://doi.org/10.1145/3318396.3318442>

## 1. INTRODUCTION

Software development is a complicated process of building computer programs that involves resources such as people, computer hardware and software. Latest trend in technology and quickly changing market requirements have resulted in more complex software systems. Software testing is significant part of software development, advancement in testing processes (such as automated testing) is gaining researchers attention [1].

Software testing ensures the system under test is build according to specification and expectations of customers. Generally, software testing is performed by executing test cases on system under test and comparing output with expected results. A test case is an artifact that includes virtual data, pre-conditions and post-conditions, expected output and any other conditions required for execution of a test case. In test design phase, test cases are derived from software requirements. Requirements are basis for design, development and testing of systems. Inconsistency, incompleteness and incorrect requirements specification are biggest challenges in requirement engineering. Ontologies provide effective means for representation of different concepts. Ontology based requirement modeling and elicitation is widely used to diminish complexity, ambiguity and difficulty in understanding requirements [2]. Goal of ontology-based requirement elicitation is to have a formal, unambiguous, reusable and extendable method to facilitate change management [3]. In software testing, test execution phase is almost automated, while contributions are being made to automate test design phase [4]. An important objective of this paper is to automatically generate sufficient amount of test cases and relevant parameter setting with limited available resources.

There is variety of techniques and approaches towards software testing. In order to narrow down the scope of the paper, we have particularly targeted black box testing. When software tester doesn't have information about internal structure of the system, system under test is depicted as a block box. Learning based software testing is an iterative approach to automate specification-based testing. In Learning based testing, model of black box system is learned from requirements written using formal language. Specification based black box testing is concerned with functional testing. Except test case generation, test execution and test oracle can be automated by implementing the criteria of comparing test case output with expected result.

In existing literature, different efforts are made by researchers to develop ontologies for different phases of software project development ranging from software requirements to software testing phase. Ontology is a knowledge-based system where domain knowledge can be represented by a formalized common conceptualization. Ontologies provide a common communication of knowledge by capturing both the structure and semantics of information environment and can be significantly helpful in understanding and sharing knowledge of complex software systems in efficient manner. For example, application specific ontologies can be used to explicitly represent specification of project to ensure consistent understating between project

stakeholders. Furthermore, ontologies can be helpful in generation of test data as they encode domain knowledge in machine interpretable format. Domain ontologies carefully derived from software requirements is a reason to extract meaningful information e.g. test cases [5].

This paper comprehensively investigates a particular solution of automatic test case generation using learning-based software testing approach based on requirement ontology. A framework is developed that uses requirement ontology to extract test case parameters and perform automatic specification based black box testing. The proposed approach has two phases: In first phase, ontology for requirement is developed to represent user requirements in the form of machine processable format. In second phase, initial test cases are generated from available domain ontologies and learning based testing approach is applied to derive counter test case examples for generating large amount of test cases automatically. Our proposed approach enables the selection and modification of test data from different ontologies of dissimilar domain by providing a consistent framework.

This paper is structured as follows. In **Section 2**, related work of this paper is discussed. **Section 3** provides the details regarding proposed framework for test case generation using learning-based testing and ontologies. **Section 4** concludes the whole research and future work is discussed. References are given in **Section 5**.

## 2. RELATED WORK

In this section, related work of the paper is divided into two categories to highlight the existing literature in automated software testing and ontologies for software testing. The details of mentioned categories are given in subsequent sections.

### 2.1. Automated Software Testing

Software testing is process of comparing output of software system with desired functionality and to uncover any possible defects before release. Testing is performed with the help of test cases that are derived from requirements to spot errors. As software systems are growing and becoming complex software testing takes a large amount of development time. A growing concern is improving the testing achievements and finding methods to automate testing process [6]. LBTest proposed in [7], is a learning-based testing tool for reactive systems. LBTest automate specification based black box testing by initially generating test cases from formal specification and then through learning after executing test cases on subject system. Incremental learning algorithm with combination of model checking is integrated with test execution process to generate model for counter examples of test cases. In another research, Debasish Kundu and Debasis Samanta [8] presented an approach of generating test cases from UML activity diagram. UML activity diagram is first annotated with test information and then activity diagram is transformed into activity graph. Test coverage criteria are considered to detect synchronization and loop faults. Not only faults are detected but location of these faults can also be determined using this approach ultimately reducing the software development time.

### 2.2. Ontologies for Software Testing

Few research studies have worked in the direction of developing ontologies to ease software testing phase. Anjalika and et al. [1] presented test case extraction based on ontology reasoning. For this purpose, requirement ontology is first proposed to represent requirement knowledge; test cases are derived from requirement based on action, object and actor. Actions related to objects and actors that are responsible for actions are transverse to extract data properties. In another paper, Hans-Jorg Hoppel and Stefan

Seedorf [5] presented possible utilization of ontologies in software engineering domain. It is analyzed that ontologies can be used to automate the test case generation process as test cases are built upon knowledge of application domain which can be effectively represented by domain ontologies.

Likewise, Valeh H. Nasser and et al. [9] proposed a method for selection of significant related test cases from knowledge base artifact in the form of ontologies. The proposed framework enables selection and de-selection of required test cases based on the pre-defined coverage criteria. Cu D. Nguyen and et al. [10] have contributed to provide a framework for selection of important test cases using ontologies for multi agent systems. Shared interaction ontology is utilized for observing the communication between agents in order to understand valid and invalid input data for test cases. Moreover, Yongbo Wang [11] presented OWL-S (Web Ontology for Web Services) based process specifications which are first transformed into Petri-Nets and formal representation of web services is obtained. Test cases for testing execution of different branches are generated using Petri-Nets topology. IOPE (input, output, preconditions and effect) ontology reasoning is performed for generating test data provided by OWL-S ontology for service functionality. In another research study it is analyzed that ontological test data representation can benefit in automated testing and easy to modify and extend test cases with programming skills [12].

Many proposed solutions use static approach for test case generation which is limited to generate large number of test case. Mostly, input constraints, pre and post conditions were only parameters considered and expandability of parameters was not taken into account. Another important limitation is ontology-based solutions for test case generation targeted specific domains using reasoning, a general solution is required applicable to different domains.

## 3. FRAMEWORK FOR TEST CASE GENERATION

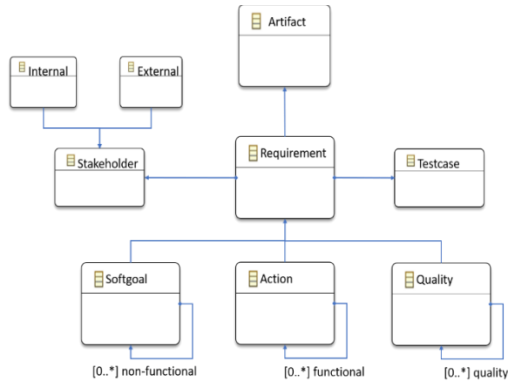
In this paper, a complete framework is proposed for test case generation by integrating two main approaches; learning based testing and ontologies.

### 3.1. Use of Ontologies in Requirement Engineering

Central to our approach, use of ontologies in software engineering is growing due to their semantic operability and machine reasoning. Researches have proposed ways to utilize ontologies in different phases of software development life cycle. Ontologies are proposed for requirement specification [2], requirement elicitation [13], testing [1] and maintenance [3] of software systems. Requirement engineering (RE) deals with identifying stakeholders need, specification and definition of desired functionalities of system.

RE generally starts from Requirement Elicitation, then Requirement Specification or Representation, Requirement Analysis and continue with Requirement Communication till the end of project. Incompletely defined, inconsistent, ambiguous and changing requirements are biggest challenges in requirement engineering. Use of ontologies in requirement elicitation process can significantly helpful in defining clear, consistent and complete requirements [14]. Requirement modeling can be greatly benefited with the use of ontologies for generating a common understanding of concepts in combination of existing modeling diagrams [2]. As test cases are derived from requirements, test cases generation [5, 9, 10, 11, 12] from requirement ontologies is gaining popularity. From these research

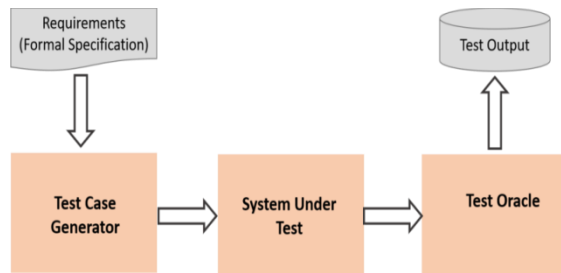
studies, it is analyzed that requirement ontology is the foundation to automatically generate test cases. In Figure 1 conceptual model of requirement ontology is given. The main classes along with their relationships are shown. A requirement may be specialized as Softgoal, Action or Quality needs. Requirement may have one or more instances of Testcase and Stakeholder class. Stakeholder can be of two types either internal or external.



**Figure 1. Proposed Requirement Ontology Model**

### 3.2.Specification Based Testing

Specification based testing mainly concerns with the functionality or behavior of a system. User requirements, constraints, pre-condition and post-conditions are represented using formal logic language. Therefore, the test case generation can be done automatically using constraint solving algorithm [15]. In Figure 2, general framework for specification-based testing is illustrated. Test case generator implements the logic of constraint solving to construct test cases from input requirements in formal logic language such as Linear Temporal Logic (LTL). The System under test is generally considered as black box, output of test case execution is compared with system requirement to make verdict about executed test case. Functional requirements of a system expressed as input constraints, pre and post-conditions are solved by constraint solver to generate test cases. Constraint solver is unable to generate sufficient amount of test cases using limited input data to satisfy functional testing of a system [7].



**Figure 2. Specification Based Testing Process**

Therefore, we need a mechanism to have a large number of test cases with virtual data to test a single functionality. Domain ontologies can assist in generating test cases and input parameters as they encode domain knowledge. Ontologies form dissimilar domain can also be helpful in generating test data as different systems have some common functionality for example login, registration and search. So, a possible solution can be to select test cases from existing dissimilar ontologies and update parameters (if required) by software tester and then apply test case generation to obtain large number of test cases against each requirement

### 3.3.Learning Based Testing

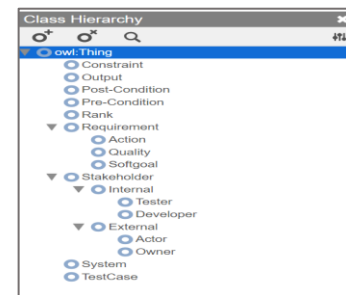
Learning based testing improves the process of specification based black box testing by adding feed back loop to testing process in order to improve the generation of test cases. Aim of learning-based testing is to generate large amount of test data by execution of existing test cases derived from formal requirements. In [4], a learning-based testing tool for reactive systems is presented. The output of test execution along with requirements is used to learn the model of black box (system under test). This learned model is then analyzed by model checking technique and counter examples are generated for existing test cases. Generated counter examples are then executed as new test cases and model of system is also tested and improved simultaneously.

The limitation of proposed framework [4] is to derive initial test case from formal requirements that require extensive knowledge and time to represent functional procedures in formal logic language. The requirements in formal logic language is hard to understand and even impossible to modify with prior programming knowledge. The derived test cases from formal requirements are also limited for initially conducting functional testing. Domain ontologies of dissimilar domain with similar modules can be utilized to initially speed up the test case generation and execution process and subsequently learning based testing can be applied to generate large amount of test cases with counter examples.

### 3.4.Ontology Based Test Case Generation

In requirement engineering, a number of ontologies are developed for representation of requirements. Ontological representation of requirement benefits in many ways, a common understanding of user needs in developed, inconsistency and incomplete requirements can be avoided and most effective of all is machine processable information [3]. Some notable contributions in ontology requirement modeling are made by [16, 17, 18]. Despite great contributions, these ontologies serve some specific aspects and not suitable for all domains.

To overcome this issue, we have developed requirement ontology to use with our test case generation framework. The proposed model of requirement ontology can be used to represent any system. W3C recommend, Web Ontology Language (OWL) is adopted to define our requirement ontology. The proposed conceptual model for requirement is depicted in Figure 1 in the form of class diagram. Major concepts are similar as presented in [13]. Requirement can be defined as Action, Quality or as a Softgoal. Action is the functionality that is required, Quality represent some constraint or condition imposed on functionality while Softgoal is used for non-functional requirements. Figure 3 illustrates class hierarchy of requirement ontology. Only generalization relationship between classes is shown.



**Figure 3. Class Hierarchy of Requirement Ontology**

Relationship between classes is implemented using data and object properties as depicted in Figure 4 (a) and (b). Ontology is

developed using web interface of Protege [19]. Requirement ontology has 20 classes, 12 object properties and 6 data properties.

As shown in Figure 5, hasTestCase relationship between TestCase and Requirement class represents associated test cases with particular requirement. TestCase is associated with Pre-Condition, Post-Condition, Output and Constraint class. Pre, Post-Conditions is used to present required input parameters and test data. Output shows the expected outcome after test case execution. Constraint shows the conditions or sequence of actions required to carry out test case execution. Pre-Conditions, Post-Conditions and Constraints are defined using Java programming language. With the help of constraint solver, if-else conditions (e.g.  $x > 10$  and  $x \leq 20$ ) in constraints can be transformed into input parameters and test data. Natural Language Processing (NLP) techniques can be applied to solve pre, post-conditions if written in natural language [1].

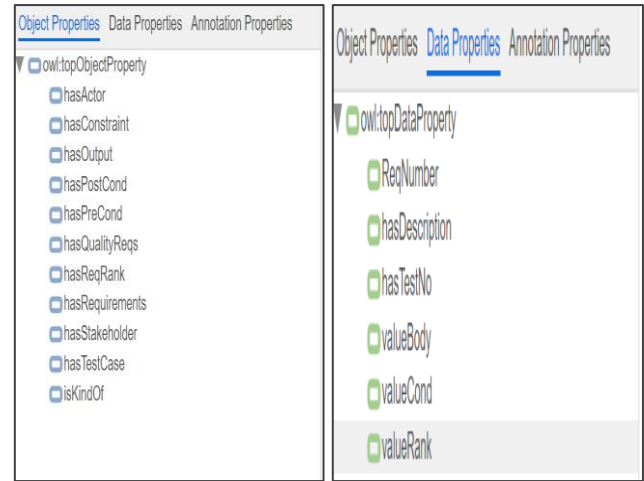


Figure 4. (a) Object Properties, (b) Data Properties

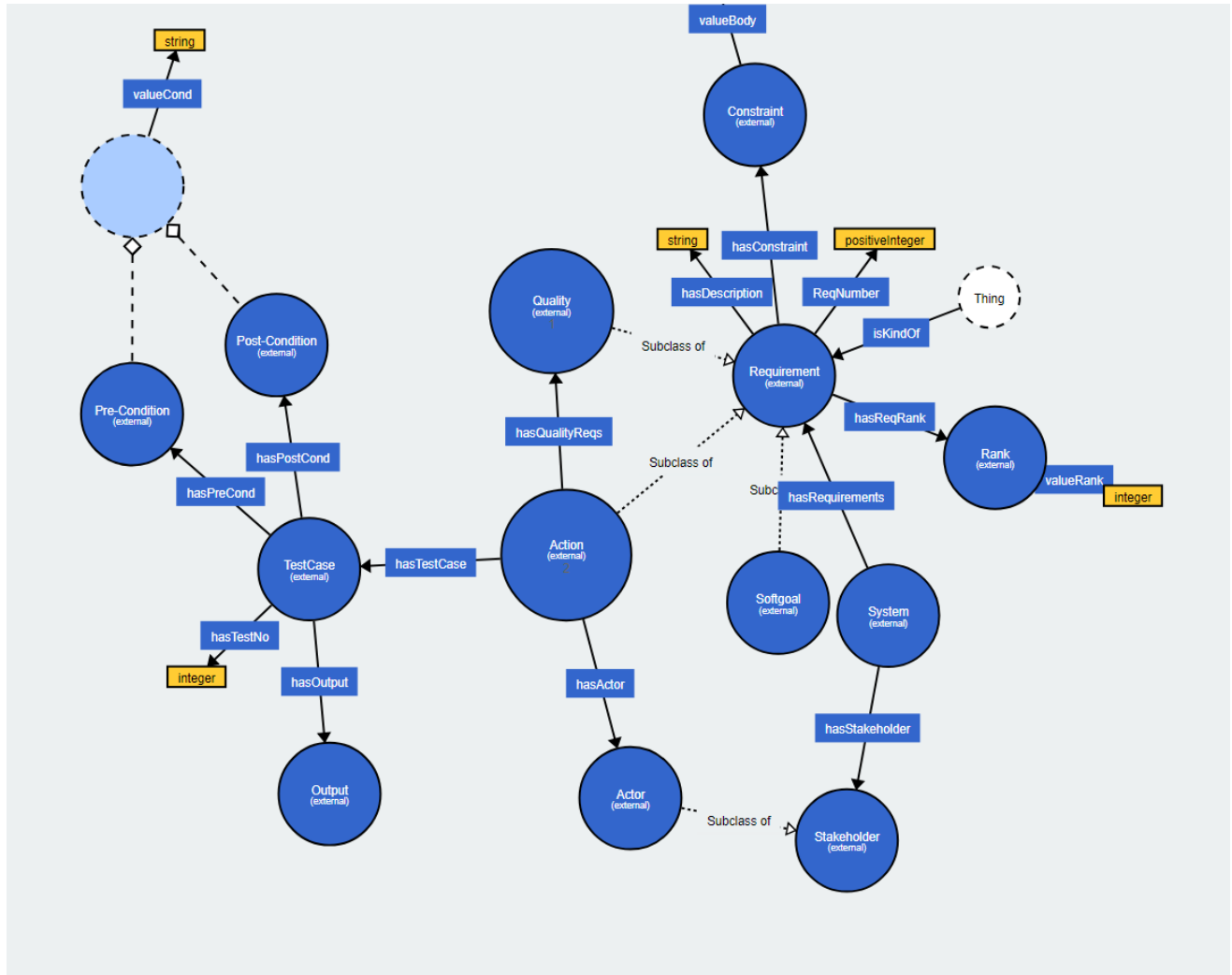


Figure 5. Graphical Representation of Requirement Ontology

### 3.5. Automatic Test Case Generation Using Ontologies

The graphical representation of the framework for automatic test case generation using learning-based testing and ontologies is illustrated in Figure 6. Knowledge engineering techniques are applied in first phase to select a set of test cases from available domain ontologies. The selected test cases may be written using natural language (i.e. English) or different programming languages (i.e. Java, Python, C sharp) according to requirement as test case come from different ontologies. The Constraint solver algorithm translates the test case conditions into test case input data, expected output and sequence of actions required to carry out test execution.

Generated test cases are not sufficient to test system completely; learning based testing is applied to infer the model of system under test by running test cases that will be used for generating new test cases. The generated model is representation of black box system under test as test has no access to internal structure of system. The model can be verified continually during testing process using model checking techniques. Based on available test data the model of system tries to find counter examples that can be executed as new test cases. To understand the framework in more details, key components are explained below. These components include dialogue manager, constraint solver, system under test, test oracle, model and model checker.

- 1) *Dialogue Manager*: It is responsible for extracting relevant information based on tester's preferences. Tester query phrases (e.g. search book, login), relevant test cases matched from existing ontologies are selected and shown. Tester can remove irrelevant test cases and edit conditions of any test case if required.
- 2) *Constraint Solver*: Constraint solver generate test cases for System Under Test by solving pre-condition and post conditions written in test cases against each functional requirement.

- 3) *System Under Test*: Software program(s) as black box represents the system under test.
- 4) *Test Oracle*: It is used to make verdicts about test case as passed or failed by comparing output with expected outcome.
- 5) *Model*: It represents the learned model of the system by analyzing input and output of test case execution process.
- 6) *Model Checker*: Model checking techniques are applied to system requirements in order to check either generated model is true representation of system or not.

### 3.6. Algorithm Execution

Step by step detail of algorithm execution is provided ranging from test case selection to test execution.

**Step 1:** Tester selects and prepares test cases written in available ontologies. Dialogue system helps in selection and pre-processing of relevant test cases.

**Step 2:** Constraint solver module solves the pre-condition, post-conditions and constraints to generate test case executable on system under test.

**Step 3:** Set of test cases is executed on system and output is produced to learning algorithm for improving model of system (initially empty). Inferred model is then satisfied by same set of test cases executed on original system. The model tries to generate counter examples for existing test cases.

**Step 4:** Counter example generated by learned model is executed. If model was trained correctly the system will fail the test. Otherwise, step 1-3 will be repeated with counter examples to infer more accurate model.

**Step 5:** Execution of learning process is terminated based on predefined criteria e.g. maximum testing time or number of test case executed.

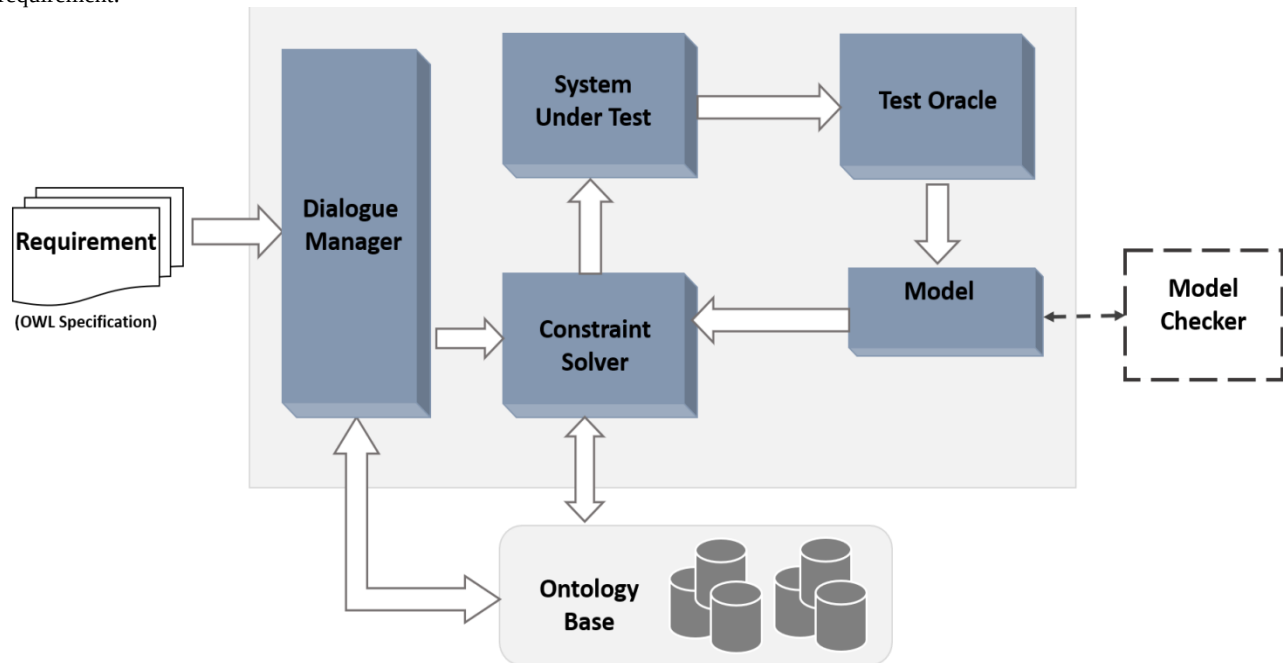


Figure 6. Overview of Framework



## 4. CONCLUSION

In this paper, we presented a complete framework for test case generation based on learning-based testing using ontologies is presented. The requirement ontology is developed using Web Ontology Language (OWL) to capture requirements in OWL specification. Use of ontologies in requirement engineering provides unambiguous, reusable and consistent means to collect, specify and communicate requirements. Requirement ontology can be instantiated from Software Requirement Specification (SRS) Document. In addition, framework for generating large amount of test case from limited input data is proposed. An iterative learning approach is used for automated specification-based black box testing. Initial test cases written by testing team using ontologies are transformed into executable format using test case generation process. Framework incorporates dialogue system to select relevant testing information from existing ontologies. Dialogue manager retrieves requirements and test data by reasoning existing ontologies. With the help of dialogue manager, existing ontologies can be used in requirement elicitation as software systems event in different domains have similar functionalities. On retrieval of information, requirement engineer can customize requirements for specific software system.

In future, we are intended to develop this framework in practice. In addition, reasoning on domain ontologies will be applied for test case generation as used in current approach for test case selection. Another future direction can be Integration of domain ontologies with requirement ontologies for automatic requirement elicitation and test case generation.

## 5. REFERENCES

- [1] Anjalika, H.N., Salgado, M.T.Y. and Siriwardhana, P.I., 2017. An Ontology Based Test Case Generation Framework (Doctoral dissertation).
- [2] Innab, N., Kayed, A. and Sajeev, A.S.M., 2012, March. An ontology for software requirements modelling. In Information Science and Technology (ICIST), 2012 International Conference on (pp. 485-490). IEEE.
- [3] Castañeda, V., Ballejos, L., Calusco, M.L. and Galli, M.R., 2010. The use of ontologies in requirements engineering. *Global Journal of Research In Engineering*, 10(6).
- [4] Meinke, K. and Sindhu, M.A., 2013, March. LBTest: a learning-based testing tool for reactive systems. In Software Testing, Verification and Validation (ICST), 2013 IEEE Sixth International Conference on (pp. 447-454). IEEE.
- [5] Happel, H.J. and Seedorf, S., 2006, November. Applications of ontologies in software engineering. In Proc. of Workshop on Semantic Web Enabled Software Engineering (SWESE) on the ISWC (pp. 5-9).
- [6] Sinha, R., Pang, C., Martínez, G.S., Kuronen, J. and Vyatkin, V., 2015, December. Requirements-aided automatic test case generation for industrial cyber-physical systems. In Engineering of Complex Computer Systems (ICECCS), 2015 20th International Conference on (pp. 198-201). IEEE.
- [7] Meinke, K., Niu, F. and Sindhu, M., 2012. Learning-based software testing: a tutorial. In *Leveraging Applications of Formal Methods, Verification, and Validation* (pp. 200-219). Springer, Berlin, Heidelberg.
- [8] Kundu, D. and Samanta, D., 2009. A novel approach to generate test cases from UML activity diagrams. *Journal of Object Technology*, 8(3), pp.65-83.
- [9] Nasser, V.H., Du, W. and MacIsaac, D., 2010. An Ontology-based Software Test Generation Framework. In *SEKE* (pp. 192-197).
- [10] Nguyen, C.D., Perini, A. and Tonella, P., 2008, May. Ontology-based test generation for multiagent systems. In *Proceedings of the 7th international joint conference on Autonomous agents and multiagent systems-Volume 3* (pp. 1315-1320). International Foundation for Autonomous Agents and Multiagent Systems.
- [11] Wang, Y., Bai, X., Li, J. and Huang, R., 2007, March. Ontology-based test case generation for testing web services. In *Autonomous Decentralized Systems, 2007. ISADS'07. Eighth International Symposium on* (pp. 43-50). IEEE.
- [12] Moser, T., Dürr, G. and Biffl, S., 2010. Ontology-Based Test Case Generation For Simulating Complex Production Automation Systems. In *SEKE* (pp. 478-482).
- [13] Zhang, X., 2011. An interactive approach of ontology-based requirement elicitation for software customization.
- [14] Siegemund, K., Thomas, E.J., Zhao, Y., Pan, J. and Assmann, U., 2011, October. Towards ontology-driven requirements engineering. In *Workshop semantic web enabled software engineering at 10th international semantic web conference (ISWC)*, Bonn.
- [15] Poston, R.M., 1997. Automating specification-based software testing. IEEE Computer Society Press.
- [16] H. Kaiya and M. Saeki, "Ontology Based Requirements Analysis: Lightweight Semantic Processing Approach," in *Proceedings of the 5th International Conference on Quality Software*, 2005
- [17] I. Jureta, J. Mylopoulos, and S. Faulkner, "Revisiting the Core Ontology and Problem in Requirements Engineering," in *Proceedings of the 16th IEEE International Requirements Engineering Conference*, 2008, pp. 71-80.
- [18] D. V. Dzung and A. Ohnishi, "Ontology-Based Reasoning in Requirements Elicitation," in *Proceedings of the 7th IEEE International Conference on Software Engineering and Formal Methods*, 2009, pp. 263-272.
- [19] Webprotege, Retrieved 15 January 2019 from <https://webprotege.stanford.edu/>